Pensamento Computacional
INF05008

Aula 15 – Alta-Ordem

Prof. Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul

Instituto de Informática

Departamento de Informática Teórica

Slides baseados no material da Profª. Leila Ribeiro.

Generalização

- Generalização é uma das técnicas fundamentais que nos permitem reutilizar soluções em outros contextos.

- Construímos uma soluções genéricas que se tornarão específicas na hora do uso.

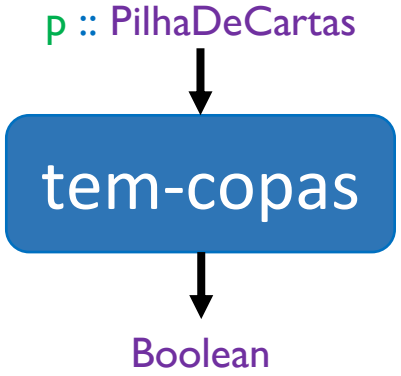
Tem Copas?

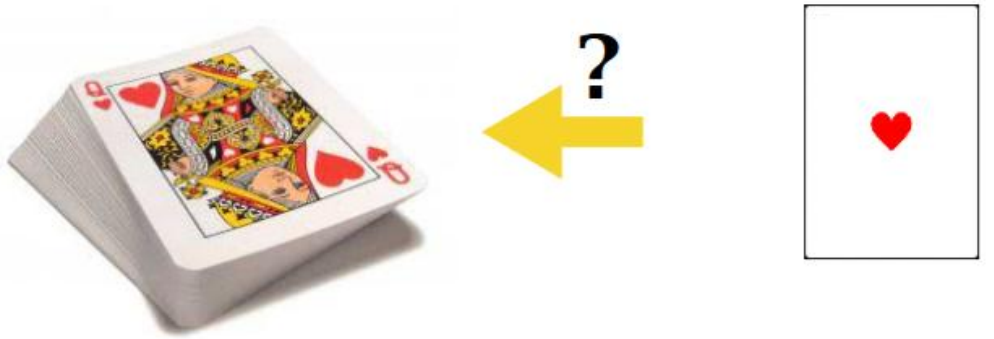
Tem Copas?

Obj: Dado uma pilha de cartas, verificar se há uma carta de copas.

Entrada: Uma pilha de cartas.

Saída: Verdadeiro caso haja carta de copas, falso caso contrário.





Conjunto Carta

```
data Carta:
    carta(
        naipe  :: String, # Naipe da carta
        numero :: Number  # Número da carta
    )
end
```

```
# Exemplos de elementos de Carta:

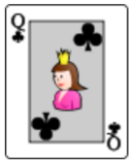
AS-COPAS = carta("copas", 1)

DEZ-OUROS = carta("ouro", 10)

RAINHA-PAUS = carta("paus", 12)
```



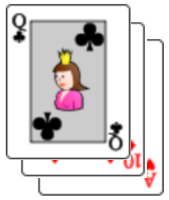




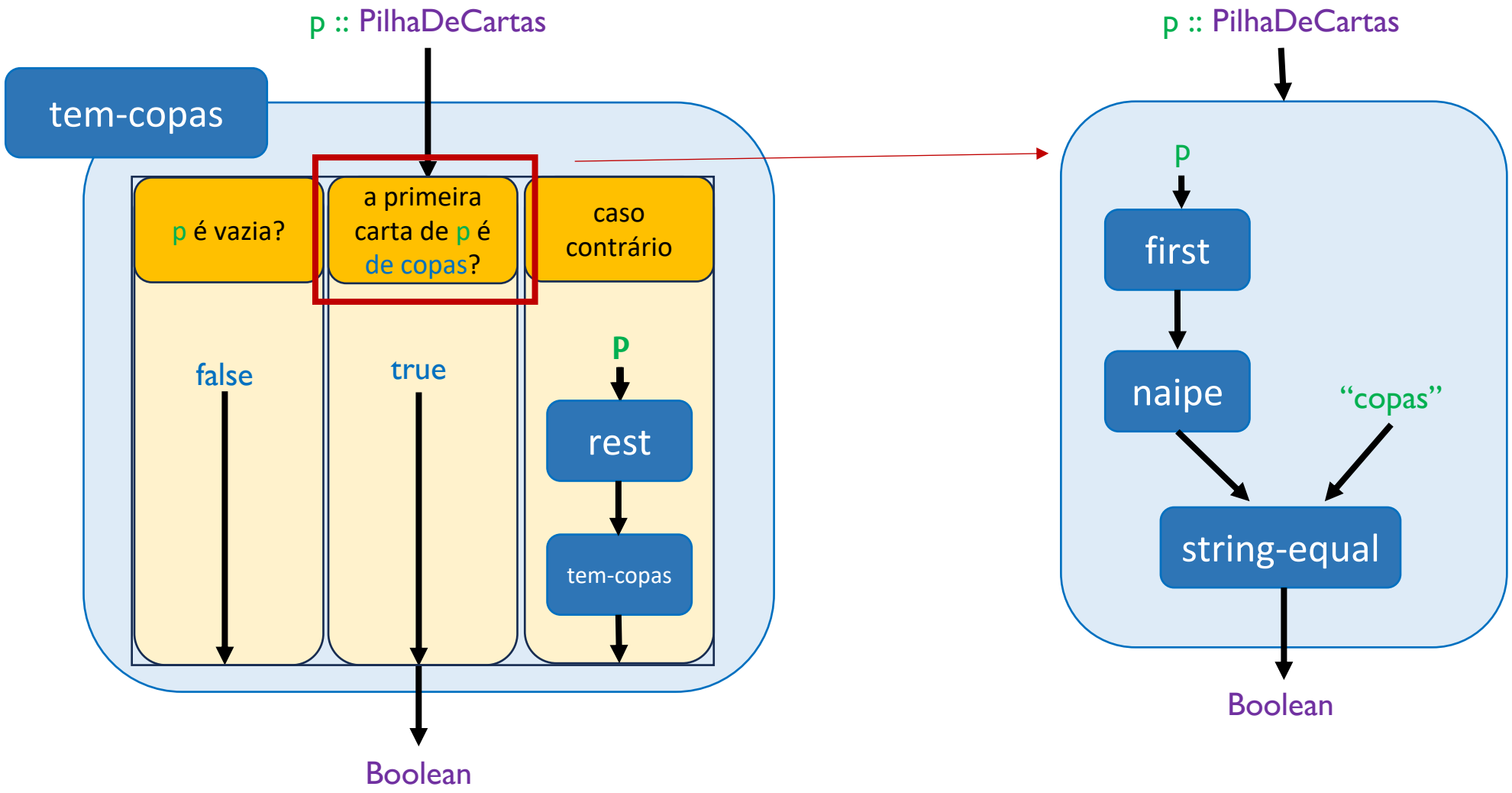
Definição de Pilha de Cartas

```
data PilhaDeCartas:
    | vazia
    | elo(first :: Carta, rest :: PilhaDeCartas)
end
```

```
elo(RAINHA-PAUS,
    elo(DEZ-OUROS,
        elo(AS-COPAS,
            vazia)))
```



tem-copas


p :: PilhaDeCartas
tem-copas
p é vazia?
a primeira carta de p é de copas?
caso contrário
false
true
p
rest
tem-copas
Boolean
p :: PilhaDeCartas
p
first
naipe
"copas"
string-equal
Boolean

tem-copas

```
fun tem-copas(p :: PilhaDeCartas) -> Boolean:
    doc: "Dado uma pilha de cartas, verifica se há uma carta de copas."
    cases (PilhaDeCartas) p:
        # Se a pilha é vazia, devolver false
        | vazia => false
        | elo(f, r) =>
            ask:
                # Se o naipe da primeira carta é copas, devolver true
                | string-equal(f.naipe, "copas") then: true
                # Senão, verificar se tem copas no resto da pilha p
                | otherwise: tem-copas(r)
            end
    end
where:
    tem-copas(elo(RAINHA-PAUS, elo(DEZ-OUROS, elo(AS-COPAS, vazia)))) is true
    tem-copas(vazia) is false
end
```

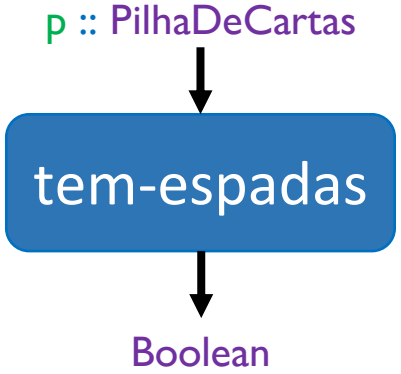
Tem Espadas?

Tem Espadas?

Obj: Dado uma pilha de cartas, verificar se há uma carta de espadas.

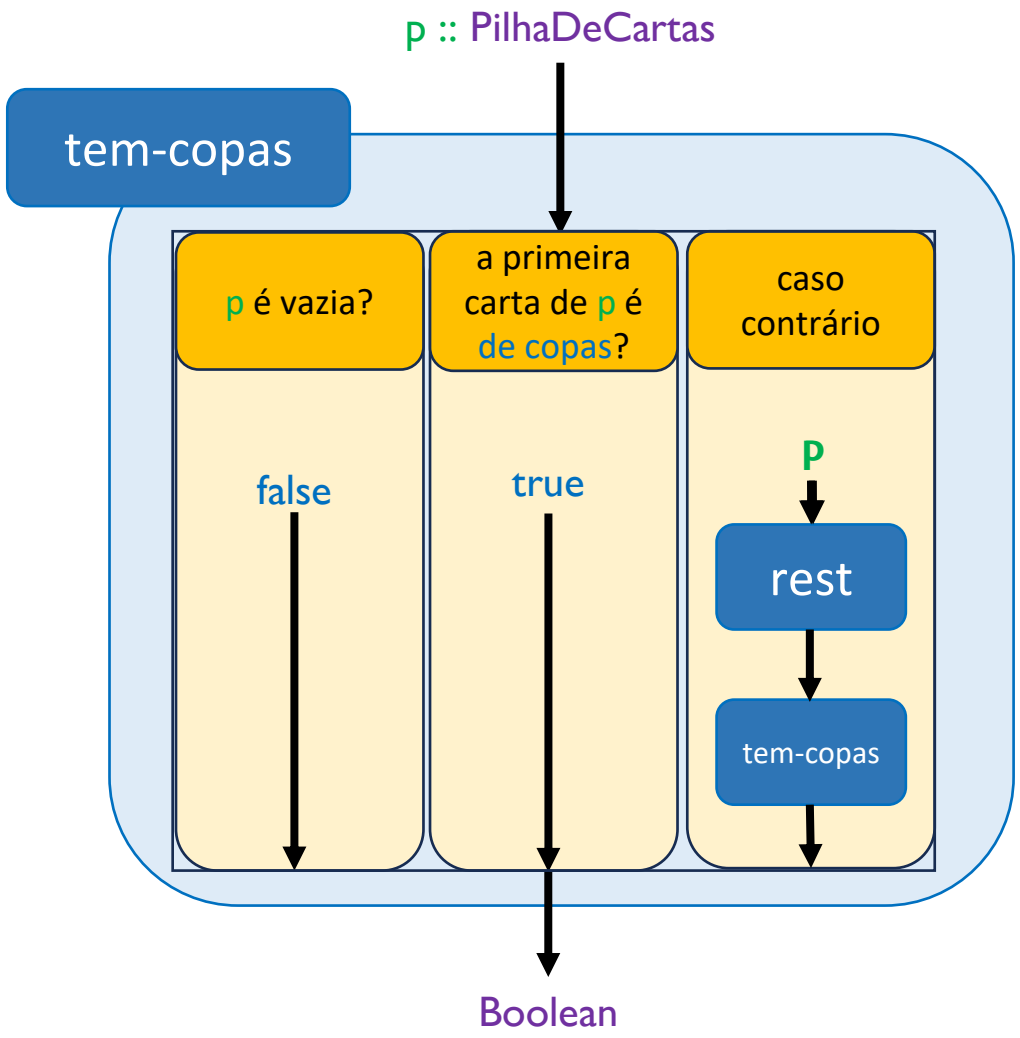
Entrada: Uma pilha de cartas.

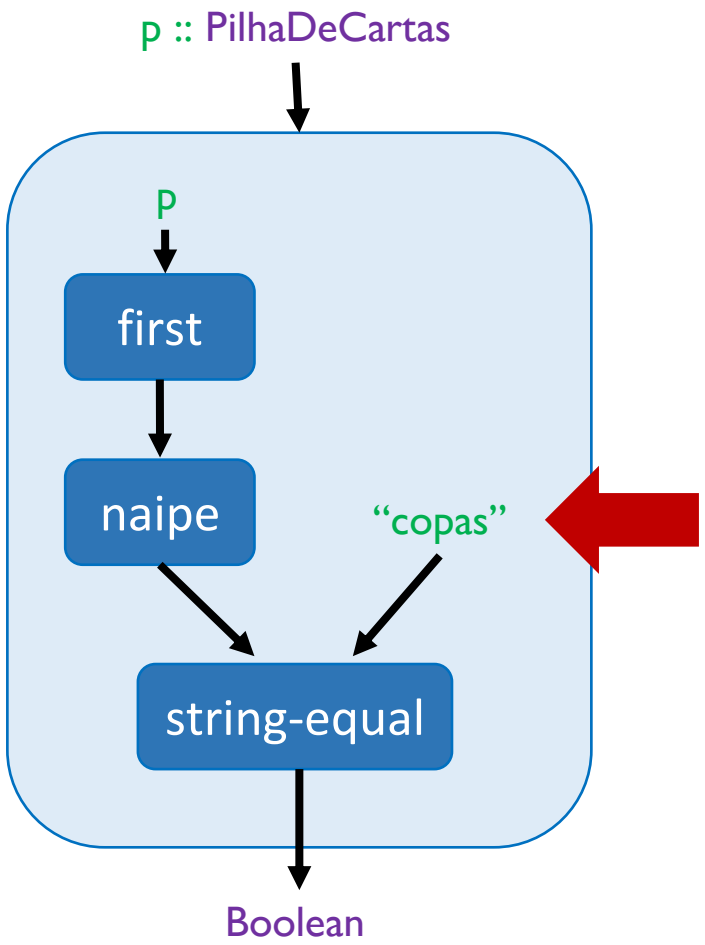
Saída: Verdadeiro caso haja carta de espadas, falso caso contrário.



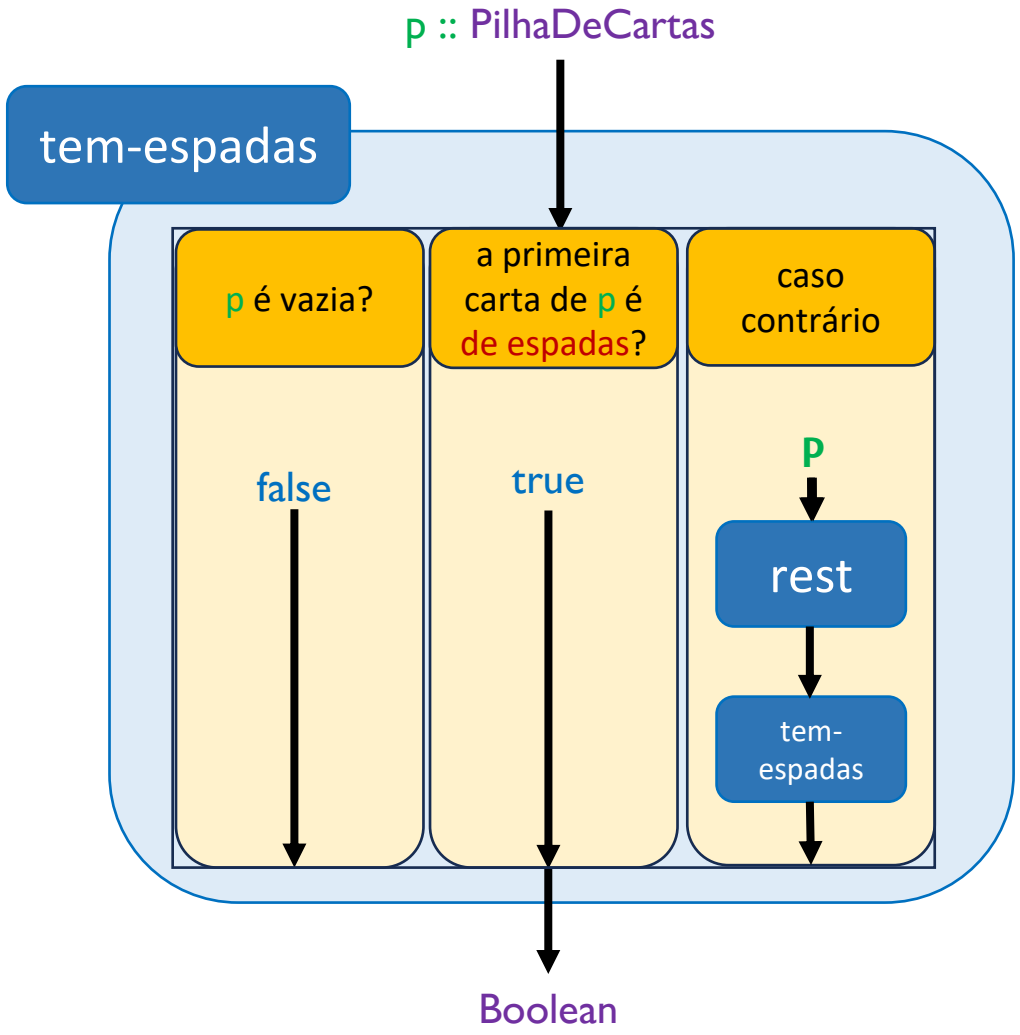


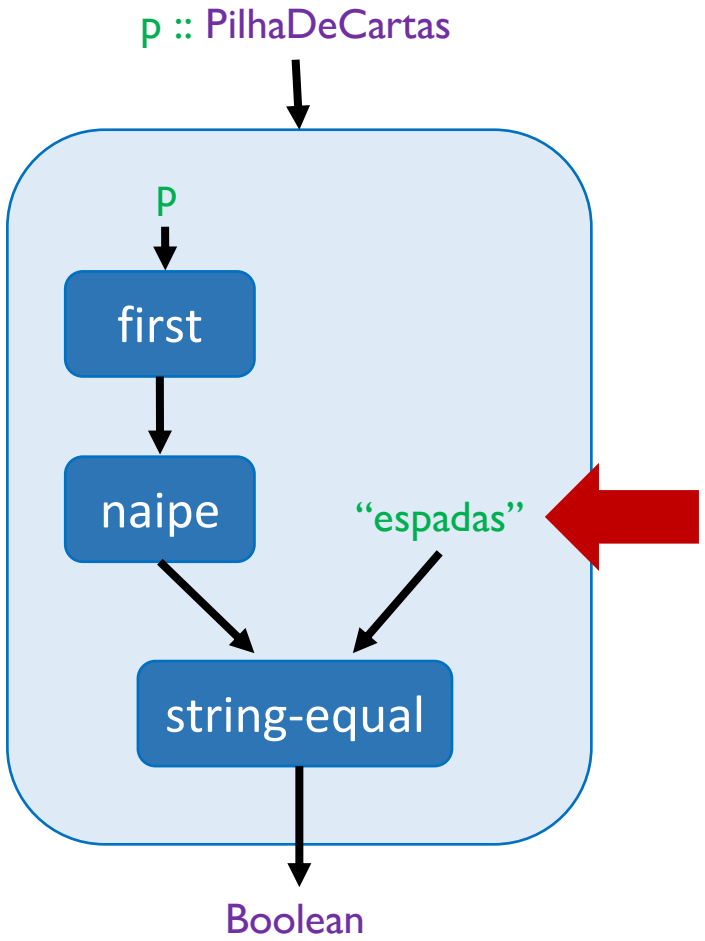
tem-copas


p :: PilhaDeCartas
tem-copas
p é vazia?
a primeira carta de p é de copas?
caso contrário
false
true
p
rest
tem-copas
Boolean

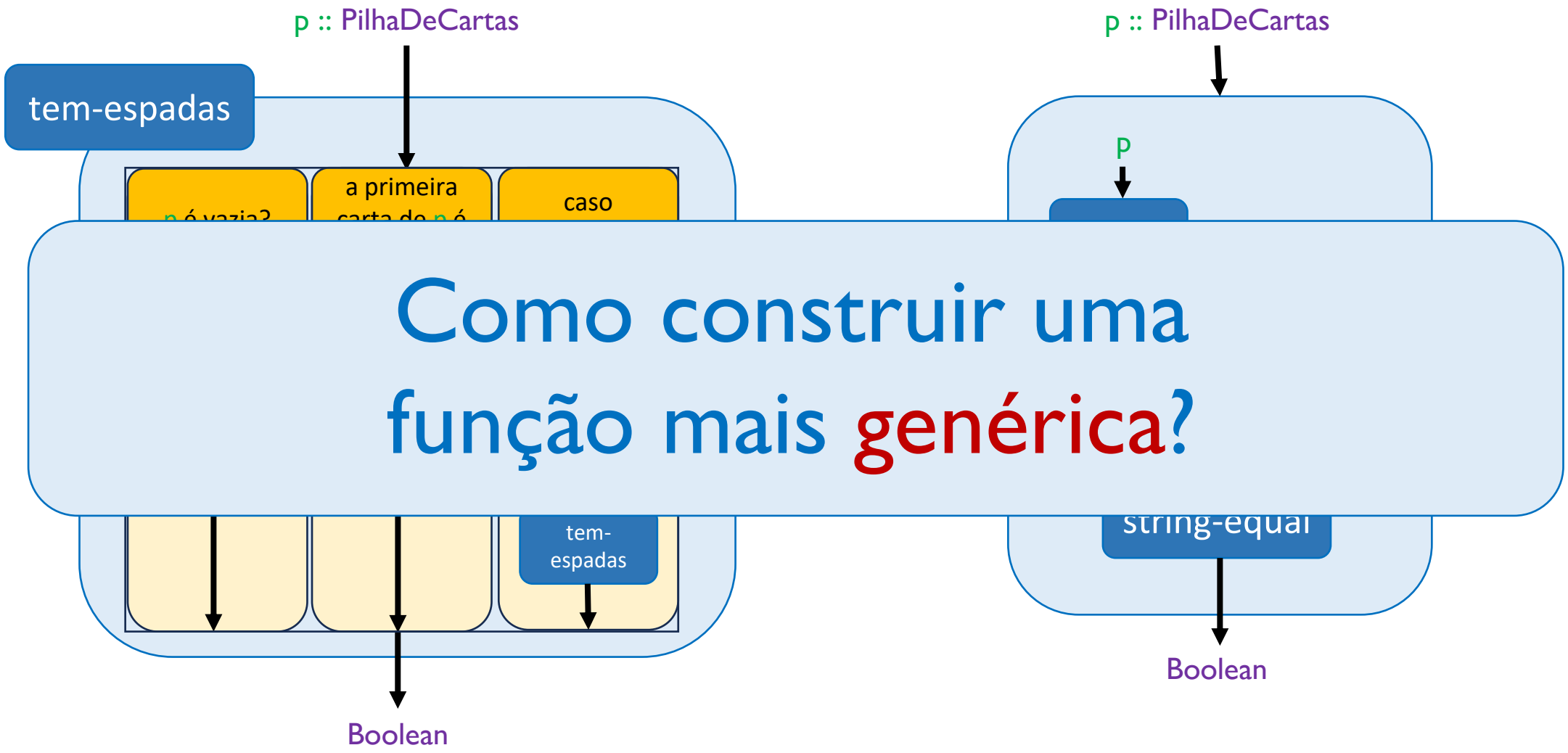

p :: PilhaDeCartas
p
first
naipe
"copas"
string-equal
Boolean

tem-espadas


p :: PilhaDeCartas
tem-espadas
p é vazia?
a primeira carta de p é de espadas?
caso contrário
false
true
p
rest
tem-espadas
Boolean


p :: PilhaDeCartas
p
first
naipe
"espadas"
string-equal
Boolean

tem-espadas



Como construir uma função mais genérica?

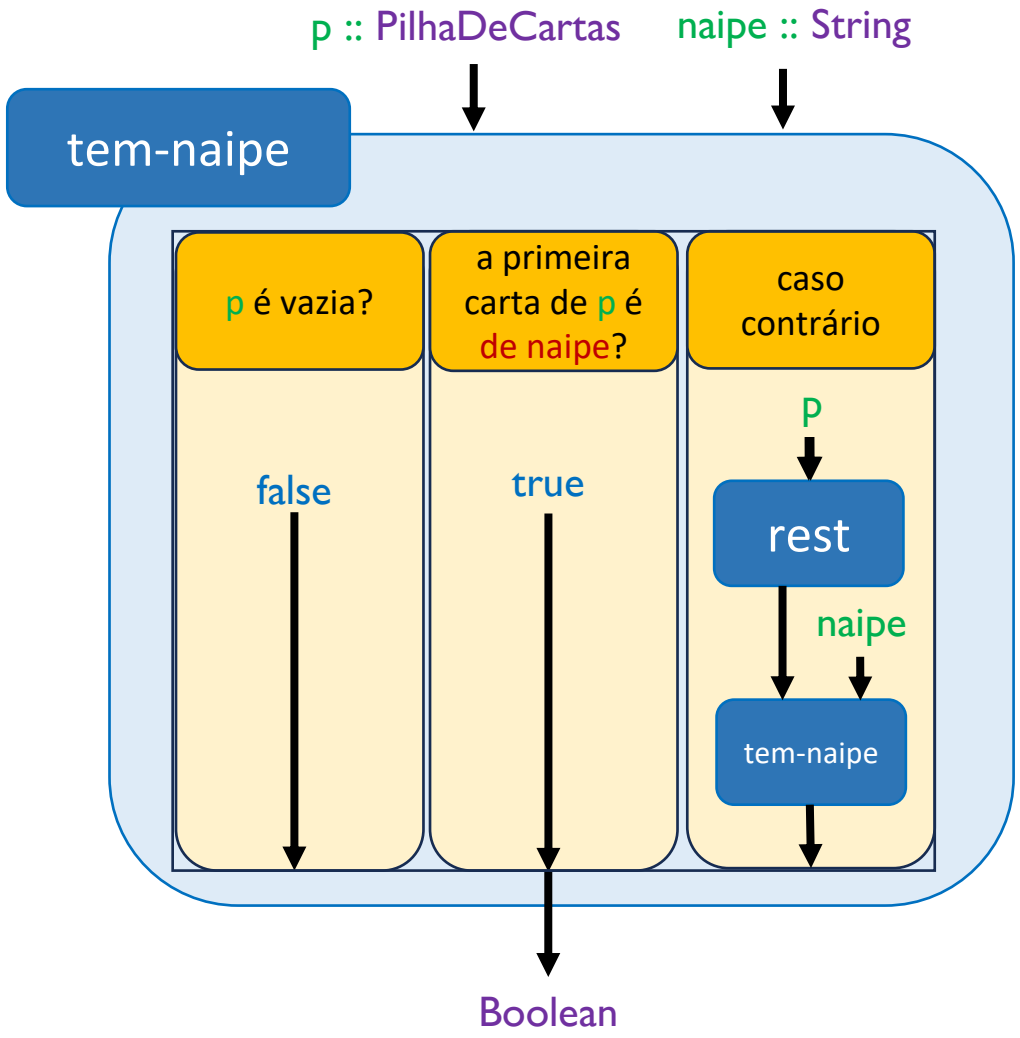
Generalização via Parâmetros

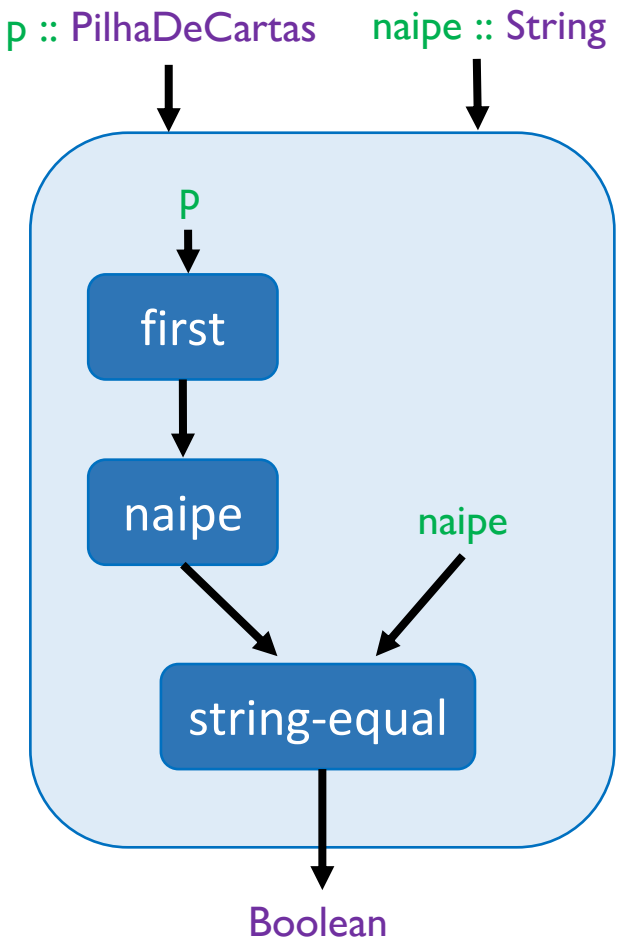
- Uma forma de tornar uma função mais genérica é através de parâmetros (argumentos) que controlam seu comportamento.

- Dependendo do valor de um determinado parâmetro, a função irá resolver uma variação diferente do problema.

P. Como generalizar tem-copas, tem-espadas, tem-ouros, tem-reis, tem-7, ... ?

tem-naipe


p :: PilhaDeCartas
naipe :: String
tem-naipe
p é vazia?
a primeira carta de p é de naipe?
caso contrário
false
true
p
rest
naipe
tem-naipe
Boolean


p :: PilhaDeCartas
naipe :: String
p
first
naipe
naipe
string-equal
Boolean

tem-copas

```
fun tem-copas(p :: PilhaDeCartas) -> Boolean:
    doc: "Dado uma pilha de cartas, verifica se há uma carta de copas."
    cases (PilhaDeCartas) p:
        # Se a pilha é vazia, devolver false
        | vazia => false
        | elo(f, r) =>
            ask:
                # Se o naipe da primeira carta é copas, devolver true
                | string-equal(f.naipe, "copas") then: true
                # Senão, verificar se tem copas no resto da pilha p
                | otherwise: tem-copas(r)
            end
    end
where:
    tem-copas(elo(RAINHA-PAUS, elo(DEZ-OUROS, elo(AS-COPAS, vazia)))) is true
    tem-copas(vazia) is false
end
```

tem-naipe

```
fun tem-naipe(p :: PilhaDeCartas, naipe :: String) -> Boolean:
    doc: "Dado uma pilha de cartas e um naipe, verifica se há uma carta deste naipe."
    cases (PilhaDeCartas) p:
        # Se a pilha é vazia, devolver false
        | vazia => false
        | elo(f, r) =>
            ask:
                # Se o naipe da primeira carta é naipe, devolver true
                | string-equal(f.naipe, naipe) then: true
                # Senão, verificar se tem naipe no resto da pilha p
                | otherwise: tem-naipe(r, naipe)
            end
    end
where:
  tem-naipe(elo(RAINHA-PAUS, elo(DEZ-OUROS, elo(AS-COPAS, vazia))), "copas") is true
  tem-naipe(vazia, "copas") is false
end
```

tem-naipe

```
fun tem-naipe(p :: PilhaDeCartas, naipe :: String) -> Boolean:
    doc: "Dado uma pilha de cartas e um naipe, verifica se há uma carta deste naipe."
    cases (PilhaDeCartas) p:
        # Se a pilha é vazia, devolver false
        | vazia => false
        | elo(f, r) =>
            ask:
                # Se o naipe da primeira carta é naipe, devolver true
                | string-equal(f.naipe, naipe) then: true
                # Senão, verificar se tem naipe no resto da pilha p
                | otherwise: tem-naipe(r, naipe)
            end
    end
where:
  tem-naipe(elo(RAINHA-PAUS, elo(DEZ-OUROS, elo(AS-COPAS, vazia))), "copas") is true
  tem-naipe(vazia, "copas") is false
end
```

Considerações

- A função tem-naipe é mais genérica que tem-copas ?
- Para obter esta generalização, tivemos que adicionar um argumento à função
- Podemos usar tem-naipe para saber se há ás de copas na pilha? Por quê?

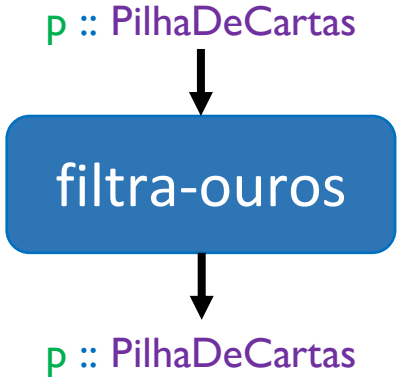
Filtra Ouros

Filtra Ouros

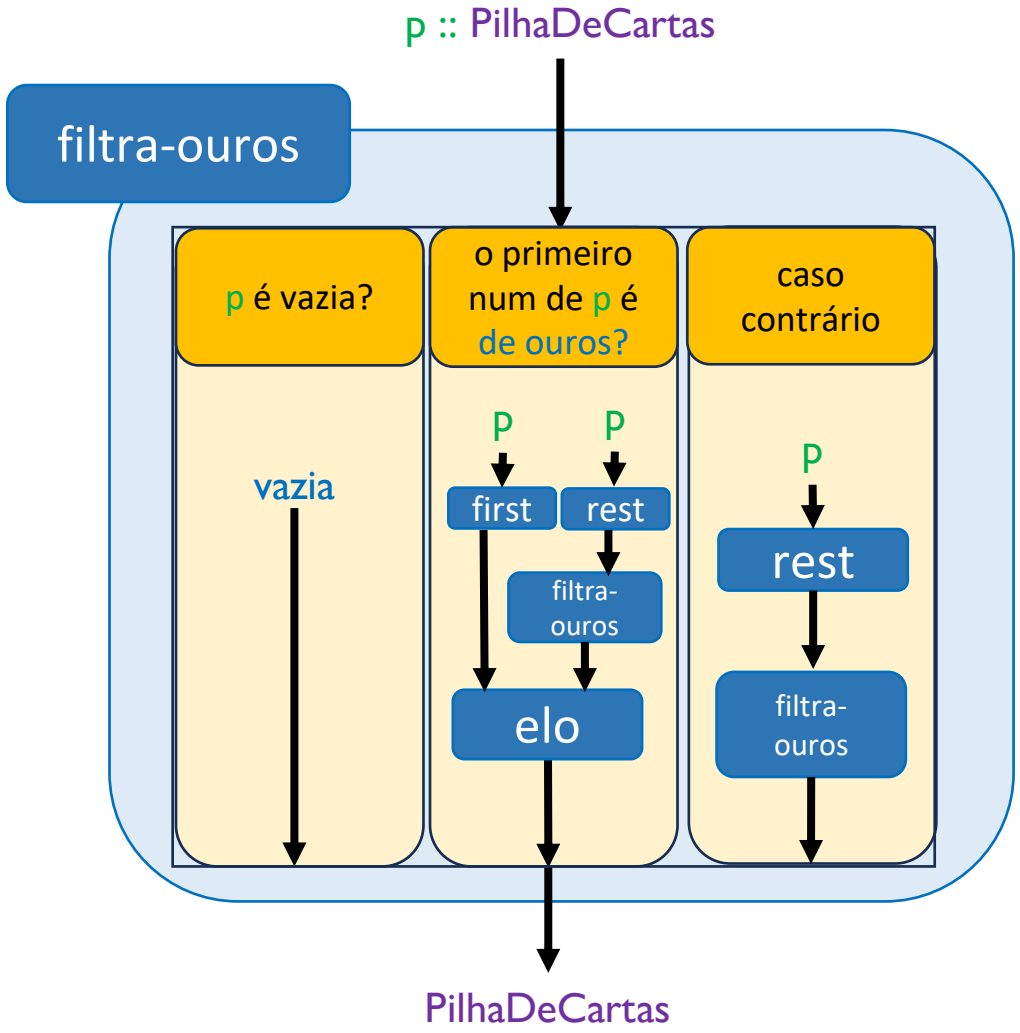
Obj: Dado uma pilha de cartas, remove todas as cartas que não são de ouros.

Entrada: Uma pilha de cartas.

Saída: Uma pilha de cartas sem cartas de ouros.



filtra-ouros




```
fun eh-ouros(c :: Carta) -> Boolean:
    doc: "Dada uma carta, verifica se o seu naipe é ouro."
    string-equal(c.naipe, "ouro")
end
```

```
fun filtra-ouros(p :: PilhaDeCartas) -> PilhaDeCartas:
    doc: "Dado uma pilha de cartas, devolve uma nova pilha
contendo apenas as cartas de ouros."
    cases (PilhaDeCartas) p:
      # Se a pilha é vazia, devolver vazia
      | vazia => vazia
      | elo(f, r) =>
        ask:
        # Se o naipe da primeira carta é ouro
          # construir uma nova pilha com esta carta
          # e com as cartas de ouros do resto da pilha p
          | eh-ouros(f) then: elo(f, filtra-ouros(r))
        # Senão, devolver as cartas de ouros do resto da pilha
          | otherwise: filtra-ouros(r)
        end
    end
end
```

filtra-ouros

```
fun filtra-ouros(p :: PilhaDeCartas) -> PilhaDeCartas:
    doc: "Dado uma pilha de cartas, devolve uma nova pilha contendo apenas as cartas de
ouros."
    cases (PilhaDeCartas) p:
      # Se a pilha é vazia, devolver vazia
      | vazia => vazia
      | elo(f, r) =>
        ask:
        # Se o naipe da primeira carta é ouro
          # construir uma nova pilha com esta carta
          # e com as cartas de ouros do resto da pilha p
          | eh-ouros(f) then: elo(f, filtra-ouros(r))
        # Senão, devolver as cartas de ouros do resto da pilha
          | otherwise: filtra-ouros(r)
        end
    end
end
```

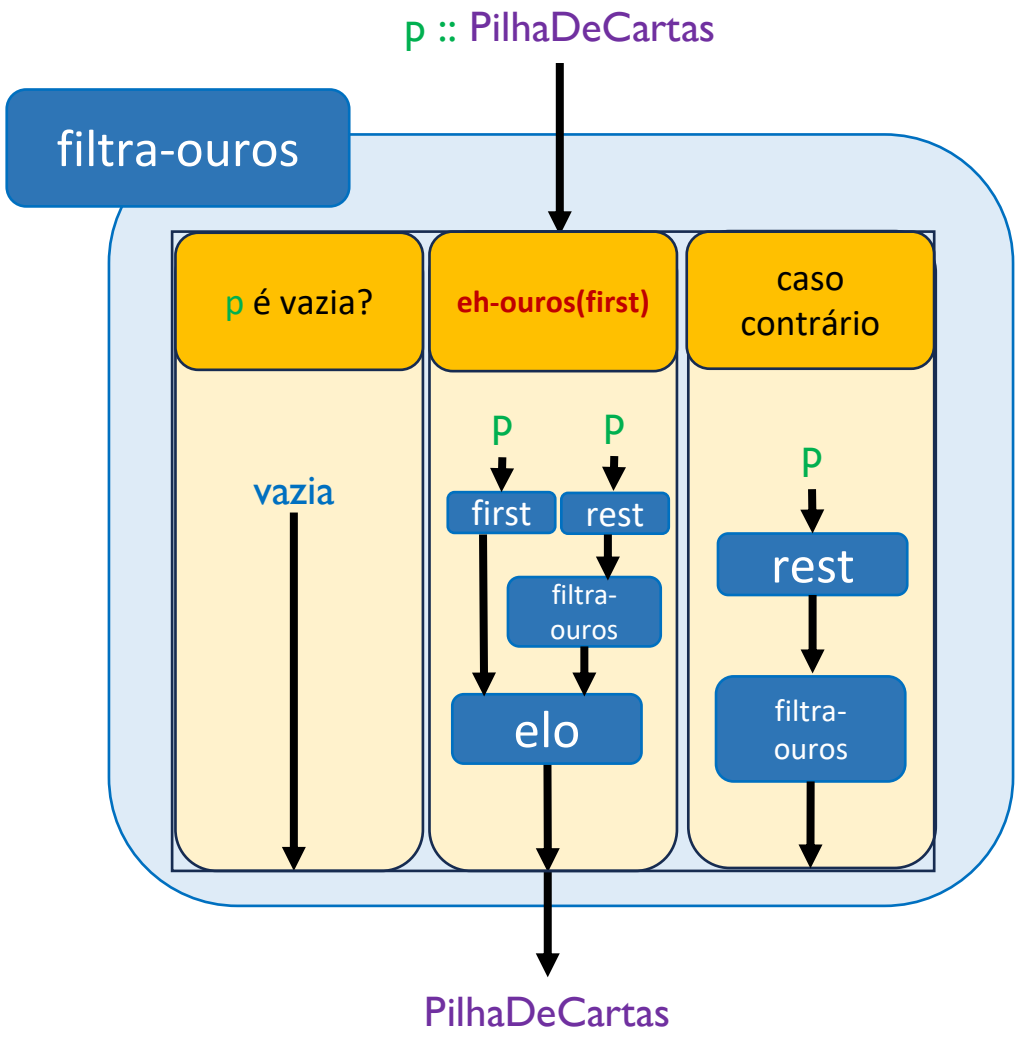
Como fazer para filtrar copas?

filtra-copas

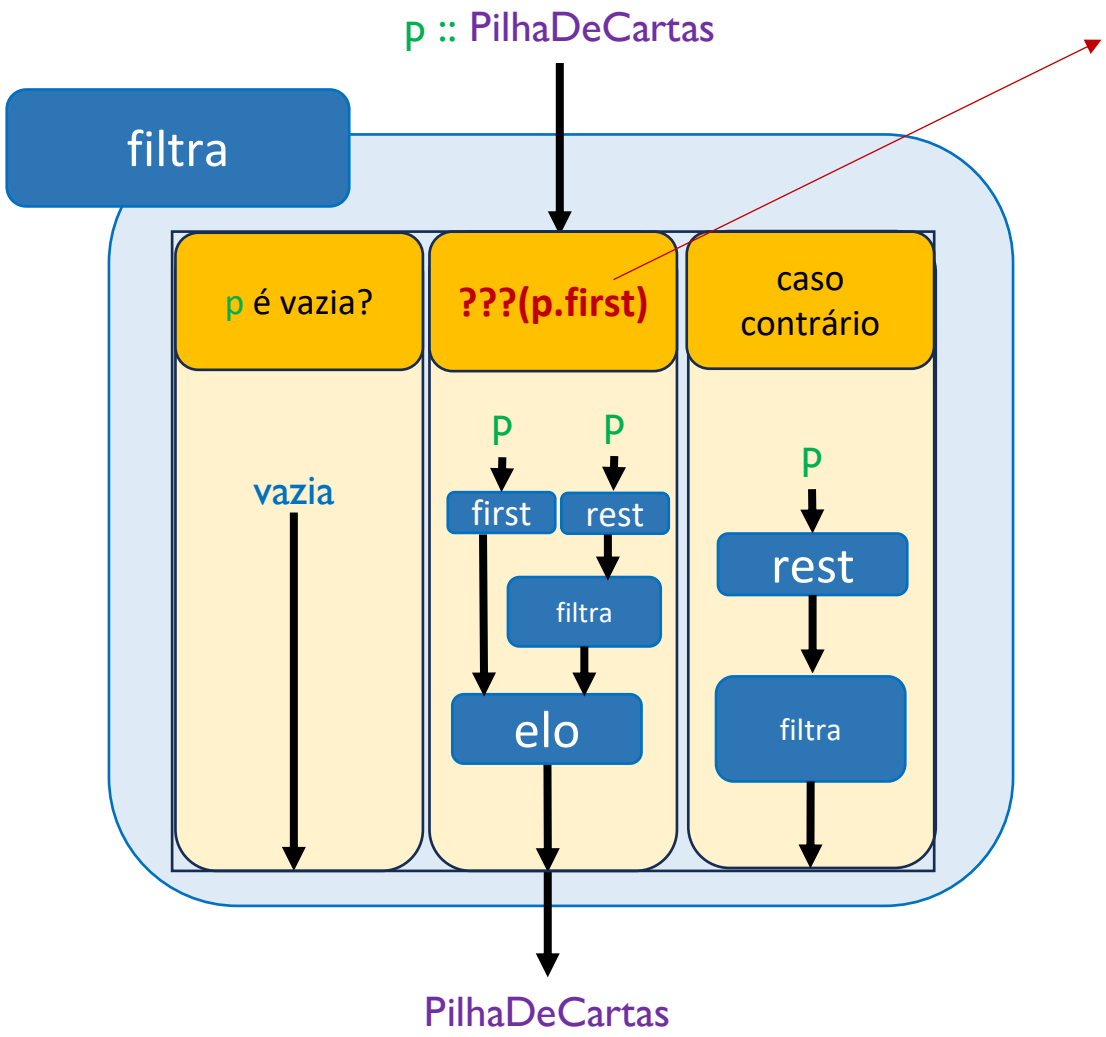
```
fun filtra-copas(p :: PilhaDeCartas) -> PilhaDeCartas:
    doc: "Dado uma pilha de cartas, devolve uma nova pilha contendo apenas as cartas de
copas."
    cases (PilhaDeCartas) p:
      # Se a pilha é vazia, devolver vazia
      | vazia => vazia
      | elo(f, r) =>
        ask:
        # Se o naipe da primeira carta é copas
          # construir uma nova pilha com esta carta
          # e com as cartas de copas do resto da pilha p
          | eh-copas(f) then: elo(f, filtra-copas(r))
        # Senão, devolver as cartas de copas do resto da pilha
          | otherwise: filtra-copas(r)
        end
    end
end
```

Como generalizar?

filtra-ouros



filtra



O que vai aqui?

eh-ouros

eh-copas

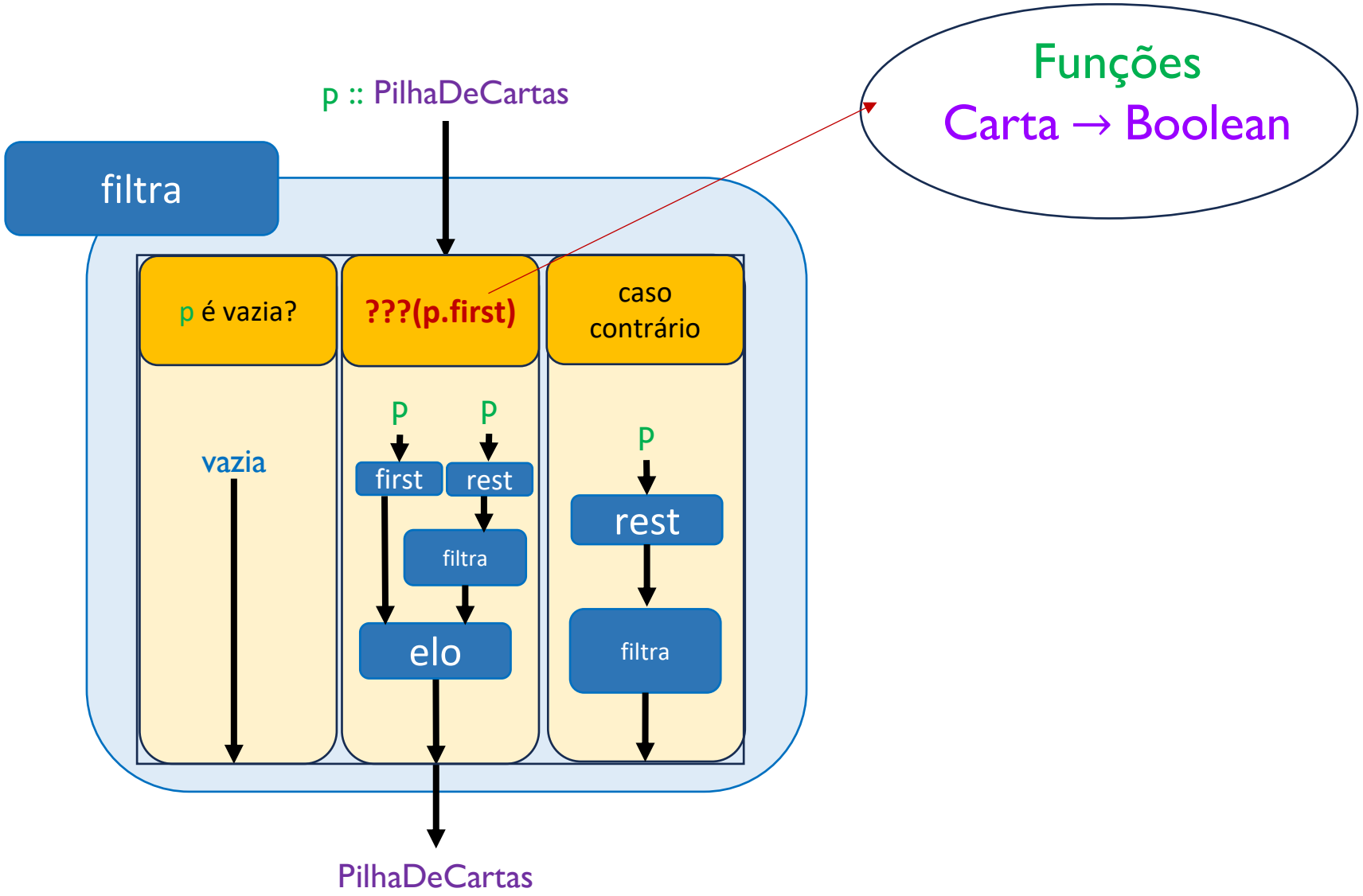
eh-ases

eh-maior-que10

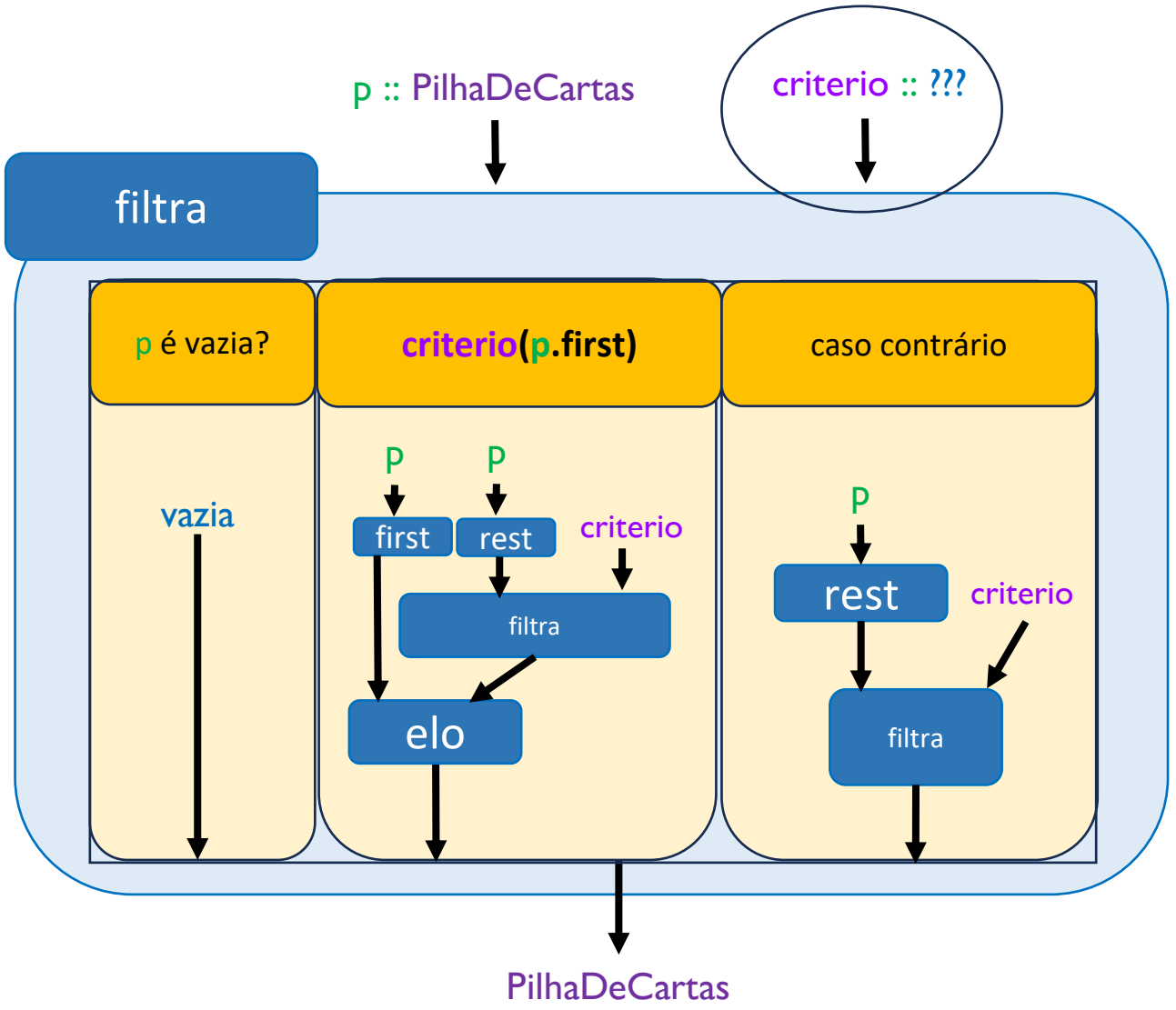
eh-rei

O que essas funções tem em comum?

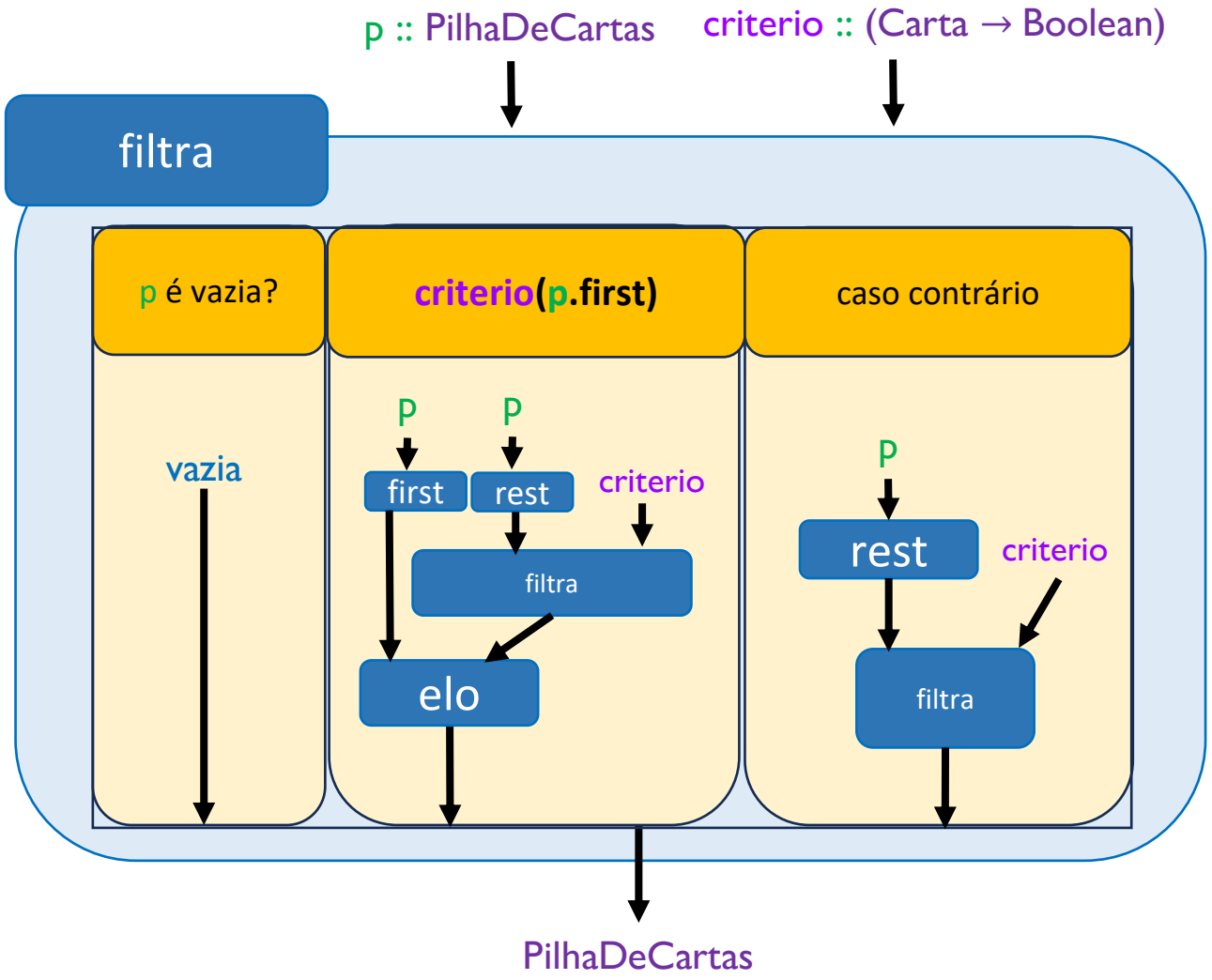
filtra


Funções
Carta → Boolean
p :: PilhaDeCartas
filtra
p é vazia?
???(p.first)
caso contrário
vazia
p
p
first
rest
filtra
elo
p
rest
filtra
PilhaDeCartas

filtra


p :: PilhaDeCartas
criterio :: ???
filtra
p é vazia?
criterio(p.first)
caso contrário
vazia
p
p
first
rest
criterio
filtra
elo
p
rest
criterio
filtra
PilhaDeCartas

filtra


p :: PilhaDeCartas
criterio :: (Carta → Boolean)
filtra
p é vazia?
criterio(p.first)
caso contrário
vazia
p
p
first
rest
criterio
filtra
elo
p
rest
criterio
filtra
PilhaDeCartas

filtra-copas

```
fun filtra-copas(p :: PilhaDeCartas) -> PilhaDeCartas:
    doc: "Dado uma pilha de cartas, devolve uma nova pilha contendo apenas as cartas de
copas."
    cases (PilhaDeCartas) p:
      # Se a pilha é vazia, devolver vazia
      | vazia => vazia
      | elo(f, r) =>
        ask:
        # Se o naipe da primeira carta é copas
          # construir uma nova pilha com esta carta
          # e com as cartas de copas do resto da pilha p
          | eh-copas(f) then: elo(f, filtra-copas(r))
        # Senão, devolver as cartas de copas do resto da pilha
          | otherwise: filtra-copas(r)
        end
    end
end
```

Como generalizar?

filtra

```
fun filtra(p :: PilhaDeCartas, criterio :: (Carta -> Boolean)) -> PilhaDeCartas:
    doc: "Dado uma pilha de cartas e um critério, devolve uma nova pilha contendo
apenas as cartas que satisfazem o critério."
    cases (PilhaDeCartas) p:
      # Se a pilha é vazia, devolver vazia
      | vazia => vazia
      | elo(f, r) =>
          ask:
             # Se a primeira carta satisfaz o critério,
              # construir uma nova pilha com esta carta
              # e com as cartas que satisfazem o critério do resto da pilha
             | criterio(f) then: elo(f, filtra(r, criterio))
             # Senão, devolver as cartas que satisfazem o critério do resto da pilha p
             | otherwise: filtra(r, criterio)
          end
    end
end
```

Funções de Alta Ordem

- filtra é uma função de alta ordem.

- Uma função de alta-ordem é uma função que tem outra função como argumento.

Comparação Filtra: Qual a mais Genérica?

```
fun filtra(p :: PilhaDeCartas, criterio :: (Carta -> Boolean)) -> PilhaDeCartas:
    cases (PilhaDeCartas) p:
      | vazia => vazia
      | elo(f, r) =>
          ask:
             | criterio(f) then: elo(f, filtra(r, criterio))
             | otherwise: filtra(r, criterio)
          end
    end
end
```

```
fun filtraX(p :: PilhaDeCartas, s :: String) -> PilhaDeCartas:
    cases (PilhaDeCartas) p:
        | vazia => vazia
        | elo(f, r) =>
            ask:
               | (s == "eh-ouros") and (f.naipe == "ouros") then: elo(f, filtraX(r, s))
               | (s == "eh-copas") and (f.naipe == "copas") then: elo(f, filtraX(r, s))
               | (s == "eh-paus") and (f.naipe == "paus") then: elo(f, filtraX(r, s))
               | (s == "eh-espadas") and (f.naipe == "espadas") then: elo(f, filtraX(r, s))
               | otherwise: filtraX(r, s)
            end
    end
end
```

Considerações

- A função filtra é mais genérica que filtraX: filtraX só consegue filtrar segundo 3 critérios pré-estabelecidos, enquanto filtra é (quase) totalmente aberta quanto ao critério.

- A restrição é que o critério de filtra deve ser uma função com contrato (Carta→Booleano).
  - Por quê?

- O uso de alta-ordem dá um grande poder de expressão para a linguagem: com poucas e pequenas funções, pode-se realizar um número enorme de tarefas.

Meta-Programação

- Meta-programação é o uso de funções de alta-ordem.
  - A ideia é brilhante e simples, mas muito poderosa.

- Esse conceito provê o fundamento para computadores de propósito geral
  - Um sistema operacional é um programa que manipula outros programas

- Meta-programação também fundamenta o estudo dos limites da Computação
  - Surgem a partir de paradoxos gerados por um programa analisar ele próprio ...

Exercícios

1. Crie uma função de alta ordem calculadora que realize operações aritméticas sobre um par de números.
2. Crie uma função mapeia que, dadas uma função unária sobre números e uma lista de números, gera uma lista com os resultados da função aplicada a cada elemento da lista original.
   Ex: mapeia(sqr, [list: 2, 5, 10, 3]) produz [list: 4, 25, 100, 9]
3. Apresente uma função filtra-num que é usada para selecionar elementos de uma lista de números. A função deve receber uma função que recebe um número e retorna um booleano e uma lista e produzir uma lista contendo somente os elementos da lista original para os quais a função fornecida retorna true. Teste a função usando uma função que verifica se um número é par.

Próxima Aula

- Funções Anônimas - Expressões Lambda

- Funções de Alta-Ordem:
  - Filter
  - Map
  - Fold

